AYUDANTÍA

PREPARACIÓN EXAMEN

IIC2143 - INGENIERÍA DE SOFTWARE

DISEÑADA POR RODOLFO ARMAS & CONSTANZA HUERTA

EJERICICIO ARQUITECTURA DE SOFTWARE

CONTEXTO

Estás diseñando una aplicación de compras online que deberá:

- Atender a millones de usuarios.
- Mantenerse fácilmente a lo largo de los años.
- Ser altamente disponible y eficiente.
- Ofrecer servicios como carrito de compras, catálogo, pagos y notificaciones.

INSTRUCCIONES

1. Elige dos arquitecturas para implementar tu solución
2. Describe cómo organizarías tu aplicación usando cada una. Considera:
    - Distribución de componentes.
    - Escalabilidad.
    - Mantenibilidad.
    - Independencia de servicios.

RESPUESTAS

Three-Tier Architecture:

- Capa cliente: navegador o app móvil.
- Capa de aplicación: lógica del negocio (carrito, pagos, etc.) en un servidor central.
- Capa de datos: base de datos única para todo.

RESPUESTAS

Microservicios:

- Cada función (carrito, pagos, catálogo, notificaciones) es un servicio independiente.
- Cada servicio puede tener su propia base de datos o almacenamiento.
- Comunicación entre servicios por HTTP/REST o mensajería (ej: Kafka).

INSTRUCCIONES

3. Completa la siguiente tabla comparativa

Aspecto	**Arquitectura - 1**	**Arquitectura - 2**
Separación de componentes		
Escalabilidad		
Facilidad de mantenimiento		
Independencia de fallos		
Complejidad de implementación		

RESPUESTAS

Aspecto	Three-Tier	Microservicios
Separación de componentes	Separación clara en capas, pero módulos fuertemente acoplados.	Separación total: cada servicio es independiente.
Escalabilidad	Escalar toda la capa de aplicación completa.	Escalabilidad por servicio según demanda.
Facilidad de mantenimiento	Difícil si hay muchos módulos en un solo bloque.	Fácil: equipos distintos pueden mantener servicios distintos.
Independencia de fallos	Un fallo puede tumbar todo el sistema.	Fallos aislados al servicio afectado.
Complejidad de implementación	Simple de montar al principio.	Requiere más planificación y herramientas (Docker, K8s).

INSTRUCCIONES

4. ¿Cuál arquitectura elegirías y por qué?

RESPUESTAS

Elegiría Microservicios, porque permite escalar y mantener servicios de forma independiente. Aunque es más complejo al inicio, es ideal para aplicaciones grandes como una tienda online que necesita alta disponibilidad, rendimiento y evolución continua sin afectar todo el sistema.

EJERCICIOS QUALITY ASSESSMENT

Parte I: Verdadero o Falso (justifica)

El test de aceptación se puede realizar sin la participación del usuario final.

Un smoke test es más detallado que un test de regresión completo.

Los tests de sistema se realizan únicamente después de los tests de aceptación.

En la estrategia de integración “Top-Down”, se utilizan drivers para simular los módulos superiores.

Respuestas Parte I: Verdadero o Falso (justifica)

F El **test de aceptación** requiere la participación de usuarios reales o representantes del cliente, ya que su objetivo es validar si el producto cumple con lo que el usuario necesita y lo que fue acordado.

F Los **smoke tests** son pruebas básicas que buscan confirmar que el sistema funciona mínimamente después de un cambio. Son **menos detallados** que los tests de regresión, que abarcan gran parte del sistema.

F Los **tests de sistema** se hacen **antes** de los de aceptación. Primero se verifica que todo funcione como fue especificado (sistema), y luego que cumpla lo que el usuario necesita (aceptación).

F En la estrategia de integración Top-Down se utilizan stubs, no drivers. Los stubs simulan módulos inferiores que aún no están disponibles. Los drivers se usan en estrategias Bottom-Up.

Parte II: Selección múltiple

1. ¿Cuál de los siguientes tests busca asegurar que un error específico fue corregido?
   A. Test de Aceptación
   B. Smoke Test
   C. Sanity Test
   D. Load Test

Parte II: Selección múltiple

1. ¿Cuál de los siguientes tests busca asegurar que un error específico fue corregido?
   A. Test de Aceptación
   B. Smoke Test
   C. Sanity Test ✅
   D. Load Test

Parte II: Selección múltiple

2. ¿Qué herramienta se usa comúnmente para automatizar pruebas de UI web?

A. Postman
B. Selenium
C. Git
D. Jenkins

Parte II: Selección múltiple

2. ¿Qué herramienta se usa comúnmente para automatizar pruebas de UI web?

A. Postman
B. Selenium ✅
C. Git
D. Jenkins

Parte II: Selección múltiple

3. ¿Cuál es una desventaja de la estrategia Big Bang en testing de integración?

A. No permite usar stubs
B. Es fácil de implementar
C. Los errores son más difíciles de aislar
D. No cubre los tests de regresión

Parte II: Selección múltiple

3. ¿Cuál es una desventaja de la estrategia Big Bang en testing de integración?

A. No permite usar stubs
B. Es fácil de implementar
C. Los errores son más difíciles de aislar ✅
D. No cubre los tests de regresión

Parte III: Preguntas abiertas

Describe cómo automatizarías un test de regresión para una aplicación web de agenda médica.

Explica con tus palabras la diferencia entre tests de integración y tests de sistema, incluyendo un ejemplo para cada uno.

¿Por qué es importante hacer pruebas de carga? ¿En qué casos no serían prioritarias?

Respuestas Parte III: Preguntas abiertas

Describe cómo automatizarías un test de regresión para una aplicación web de agenda médica.

Para automatizar un test de regresión:

- Usaría una herramienta como Selenium para simular acciones del usuario sobre la interfaz (agendar, editar, cancelar citas).
- Prepararía un conjunto de pruebas que incluya funciones clave: crear cita, buscar pacientes, generar reportes.
- Haría que estos tests corran automáticamente cada vez que se hace un cambio en el código (integración continua).
- Validaría que, tras agregar nuevas funciones (ej. notificaciones por email), el resto de las funcionalidades no se haya roto.

Respuestas Parte III: Preguntas abiertas

Explica con tus palabras la diferencia entre tests de integración y tests de sistema, incluyendo un ejemplo para cada uno.

- Los tests de integración verifican que varios componentes trabajen bien juntos. Por ejemplo, probar que cuando un usuario se registra, se guarde en la base de datos y se le envíe un correo de bienvenida. Aquí se prueba la interacción entre el sistema de usuarios y el sistema de emails.
- Los tests de sistema evalúan todo el sistema funcionando como un todo, revisando que cumpla los requisitos completos. Por ejemplo, que un usuario pueda agendar una cita médica desde el inicio hasta el final del flujo, incluyendo login, selección de doctor, fecha, y confirmación.

Respuestas Parte III: Preguntas abiertas

¿Por qué es importante hacer pruebas de carga?
¿En qué casos no serían prioritarias?

- Las pruebas de carga son importantes para asegurar que el sistema puede manejar la cantidad esperada de usuarios o datos sin fallar. Ayudan a evitar colapsos en momentos de alta demanda, como lanzamientos o eventos especiales.
- No serían prioritarias en sistemas internos con pocos usuarios o prototipos que no se usan en producción. Por ejemplo, una aplicación de uso exclusivo para 5 personas en una empresa pequeña.

Respuestas Parte I: Verdadero o Falso (justifica)

F El **test de aceptación** requiere la participación de usuarios reales o representantes del cliente, ya que su objetivo es validar si el producto cumple con lo que el usuario necesita y lo que fue acordado.

F Los **smoke tests** son pruebas básicas que buscan confirmar que el sistema funciona mínimamente después de un cambio. Son **menos detallados** que los tests de regresión, que abarcan gran parte del sistema.

F Los **tests de sistema** se hacen **antes** de los de aceptación. Primero se verifica que todo funcione como fue especificado (sistema), y luego que cumpla lo que el usuario necesita (aceptación).

F En la estrategia de integración Top-Down se utilizan stubs, no drivers. Los stubs simulan módulos inferiores que aún no están disponibles. Los drivers se usan en estrategias Bottom-Up.

EJERCICIOS SCRUM

Parte I: Verdadero o Falso (justifica)

El Product Owner asigna las tareas del Sprint Backlog a cada miembro del equipo.

El Scrum Master debe intervenir directamente en el desarrollo del producto.

El Product Backlog puede cambiar durante el desarrollo del proyecto.

En un Daily Scrum, se discuten soluciones técnicas en profundidad.

Respuestas Parte I: Verdadero o Falso (justifica)

F En SCRUM las tareas no se asignan. Cada miembro del equipo elige voluntariamente qué tareas tomará del Sprint Backlog.

F El Scrum Master no desarrolla, su rol es facilitar el proceso, eliminar impedimentos y asegurar que se sigan las prácticas de SCRUM. No debe intervenir directamente en el desarrollo del producto.

V El Product Backlog es dinámico, puede y debe cambiar constantemente según se aprenden cosas nuevas o cambian las prioridades del negocio.

F En el Daily Scrum no se discuten detalles técnicos, solo se responde brevemente qué se hizo, qué se hará y si hay impedimentos. Las conversaciones técnicas se hacen después si es necesario.

Parte II: Selección múltiple

1. ¿Cuál de estas funciones corresponde al Scrum Master?
   A. Priorizar el Product Backlog
   B. Asignar tareas al equipo
   C. Eliminar impedimentos
   D. Aceptar el resultado del Sprint

Parte II: Selección múltiple

1. ¿Cuál de estas funciones corresponde al Scrum Master?
   A. Priorizar el Product Backlog
   B. Asignar tareas al equipo
   C. Eliminar impedimentos ✅
   D. Aceptar el resultado del Sprint

Parte II: Selección múltiple

2. ¿Cuál NO es un artefacto de SCRUM?
 A. Product Backlog
 B. Sprint Planning
 C. Sprint Backlog
 D. Incremento

Parte II: Selección múltiple

2. ¿Cuál NO es un artefacto de SCRUM?
 A. Product Backlog
 B. Sprint Planning ✅
 C. Sprint Backlog
 D. Incremento

Parte II: Selección múltiple

3. ¿Cuál es el objetivo del Sprint Review?
   A. Evaluar el rendimiento individual del equipo
   B. Ajustar el Sprint Backlog
   C. Mostrar el incremento al cliente y recibir feedback
   D. Analizar qué salió mal en el proceso

Parte II: Selección múltiple

3. ¿Cuál es el objetivo del Sprint Review?
   A. Evaluar el rendimiento individual del equipo
   B. Ajustar el Sprint Backlog
   C. Mostrar el incremento al cliente y recibir feedback ✅
   D. Analizar qué salió mal en el proceso

Parte III: Preguntas abiertas

Explica la diferencia entre el Product Backlog y el Sprint Backlog.

¿Qué significa que el equipo de desarrollo sea autoorganizado en SCRUM?

¿Por qué es importante realizar una retrospectiva al final del Sprint?

Respuestas Parte III: Preguntas abiertas

Explica la diferencia entre el Product Backlog y el Sprint Backlog.

- El Product Backlog es una lista priorizada y general de todo lo que se quiere desarrollar en el producto. Es mantenida por el Product Owner y se actualiza constantemente.

- El Sprint Backlog es un subconjunto del Product Backlog que el equipo selecciona para trabajar durante un Sprint. Contiene las tareas específicas que se van a realizar en ese ciclo.

Respuestas Parte III: Preguntas abiertas

¿Qué significa que el equipo de desarrollo sea autoorganizado en SCRUM?

Significa que no hay jefes ni asignación externa de tareas. El equipo decide internamente quién hace qué, cómo se estructura y cómo cumplir con los objetivos del Sprint. Esto promueve la autonomía, la responsabilidad compartida y la colaboración.

Respuestas Parte III: Preguntas abiertas

¿Por qué es importante realizar una retrospectiva al final del Sprint?

- La retrospectiva permite al equipo reflexionar sobre el proceso de trabajo, identificar qué funcionó bien y qué se puede mejorar.
- Su objetivo es mejorar continuamente la forma de trabajar, aumentar la eficiencia y fortalecer la colaboración interna en los siguientes sprints.

MATERIA

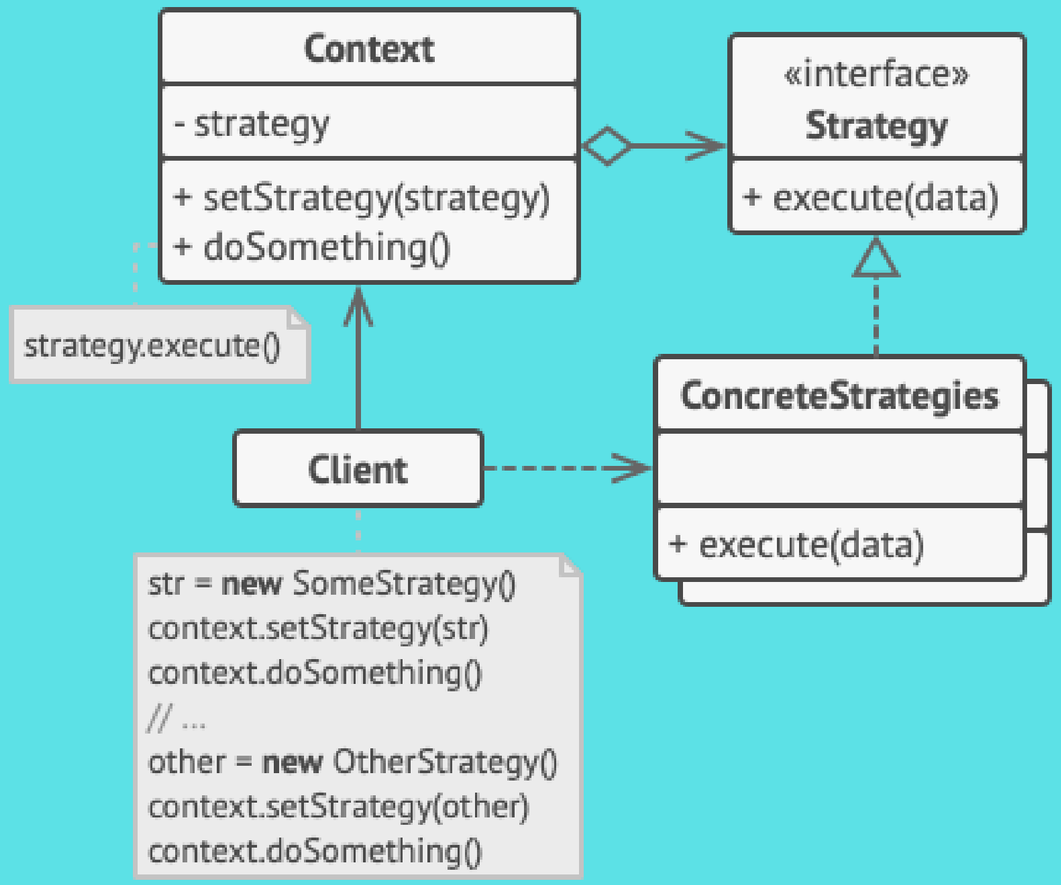
PATRONES DE DISEÑO

¿CUALES HAY?

- **Patrones de Comportamiento**
  - Strategy
  - Template Method
  - Observer
- **Patrones Estructurales**
  - Proxy
  - Adapter
  - Decorator
- **Patrones Creacionales**
  - Abstract Factory
  - Factory
  - Builder
  - Singleton

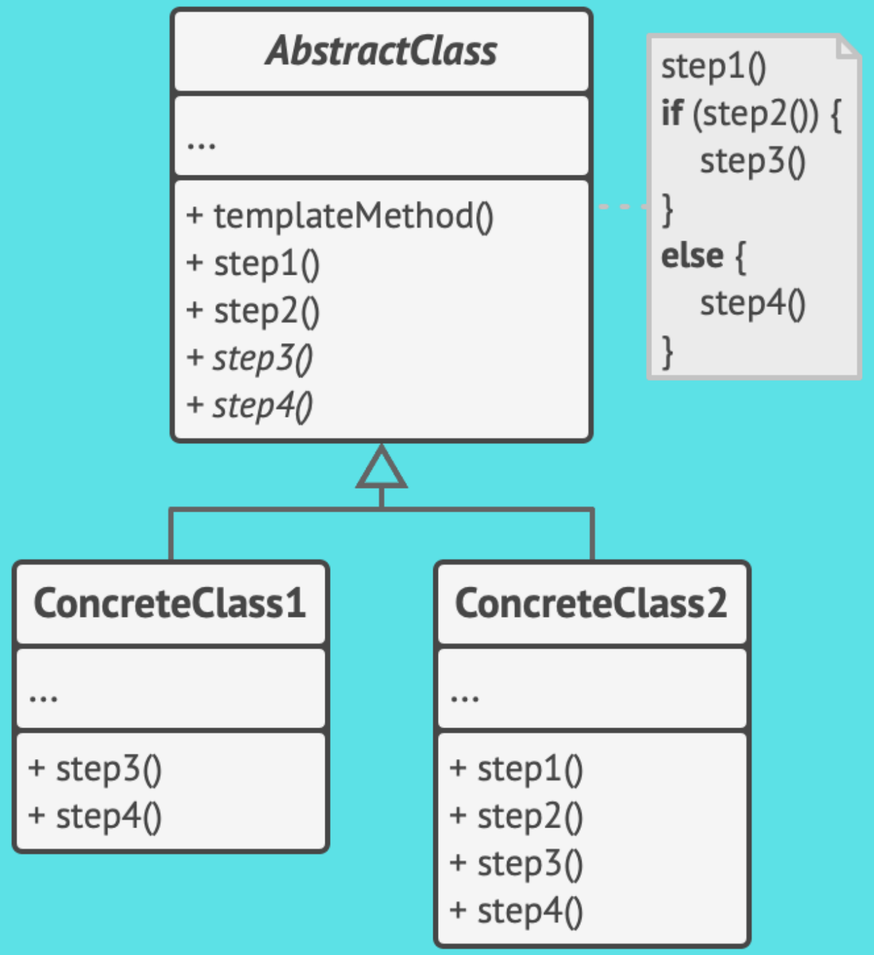
STRATEGY

- El **patrón Strategy** permite hacer una misma tarea de distintas formas.
- Cada forma diferente de hacerlo se pone en una **clase distinta**. A esas clases se les llama **strategies o estrategias** concretas.
- Existe una clase principal llamada **contexto (context)**. Esa clase no hace el trabajo, solo elige y usa la estrategia que se le indique.
- Para que el contexto pueda funcionar con cualquier estrategia, todas las estrategias deben seguir una **misma estructura**, definida en una interfaz (como un “molde” o “plantilla”).
- Las estrategias concretas **heredan** esa interfaz, lo que permite que el contexto las use sin importar cuál sea.



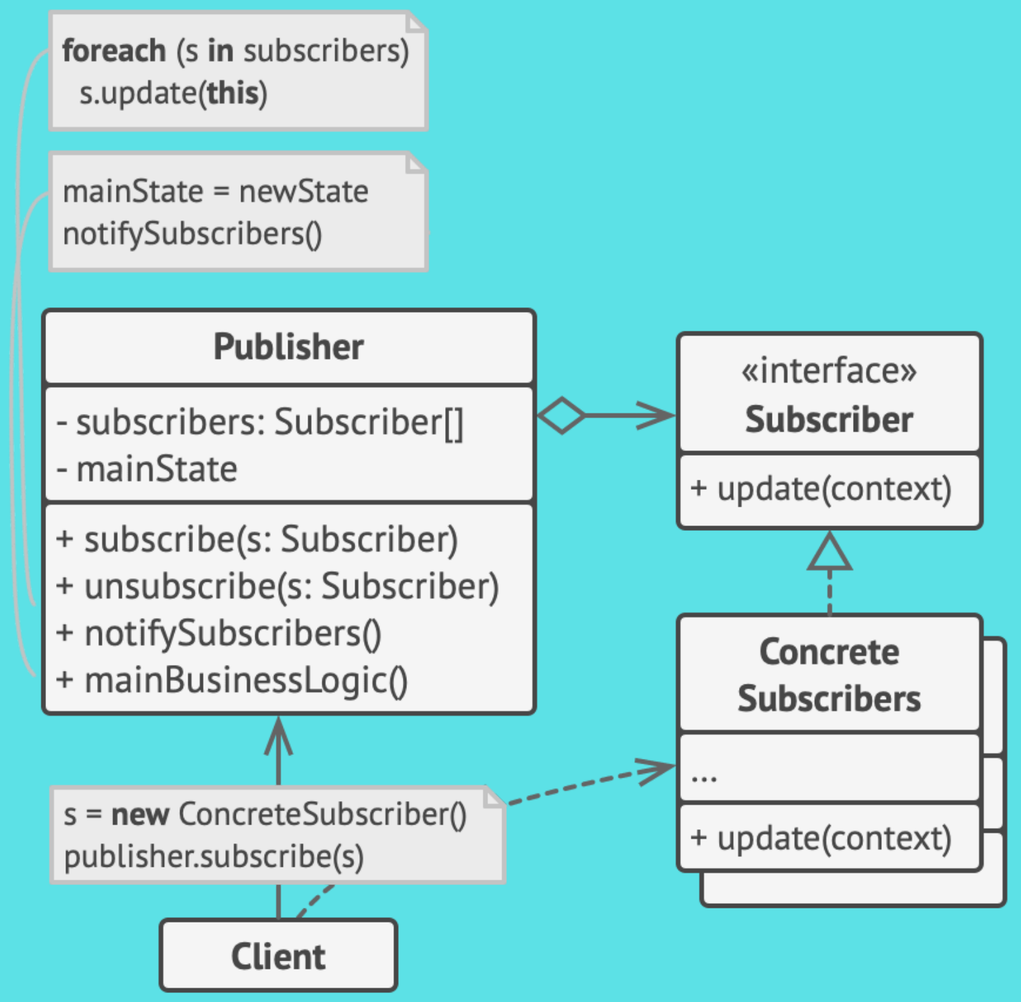
TEMPLATE METHOD

- Se usa cuando un algoritmo siempre tiene los mismos pasos, pero cada subclase puede hacer esos pasos de forma diferente.
- La clase principal define el “esqueleto” del algoritmo en un solo método **(llamado método plantilla). (No olvidar!)**
- Los pasos del algoritmo se escriben como **métodos vacíos o estándar que las subclases pueden completar o modificar.**
- El cliente (programador) debe crear su propia subclase para:
  - Implementar los pasos obligatorios (abstractos).
  - Sobre-escribir pasos opcionales, si quiere cambiar su comportamiento.
  - Nunca debe modificar el método plantilla que define el orden general de los pasos.
- Así se mantiene una estructura fija, pero con flexibilidad en los detalles.



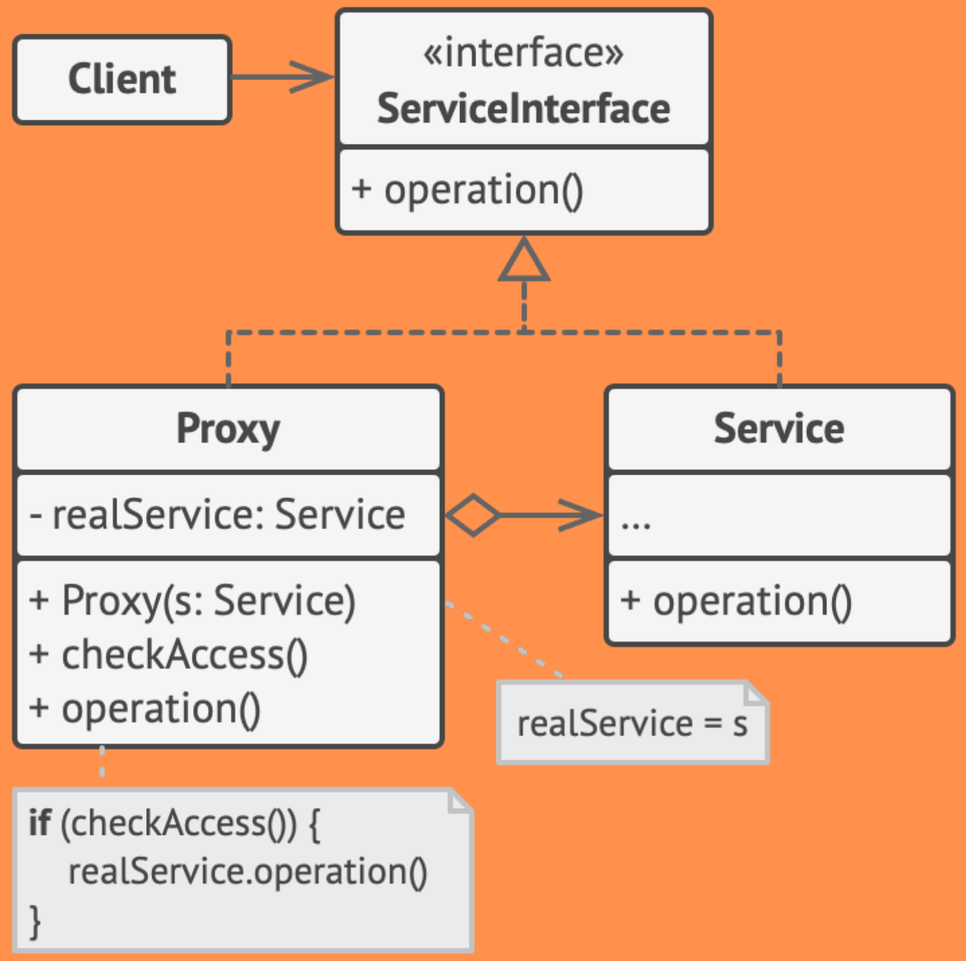
OBSERVER

- Permite que un objeto notificador envíe eventos importantes a sus suscriptores.
- La interfaz suscriptora declara el método que usará el notificador para informar.
- Los suscriptores concretos implementan esta interfaz.
- Cada suscriptor define cómo reaccionar cuando recibe una notificación.
- Así, el notificador no necesita saber qué hacen los suscriptores, solo les avisa.



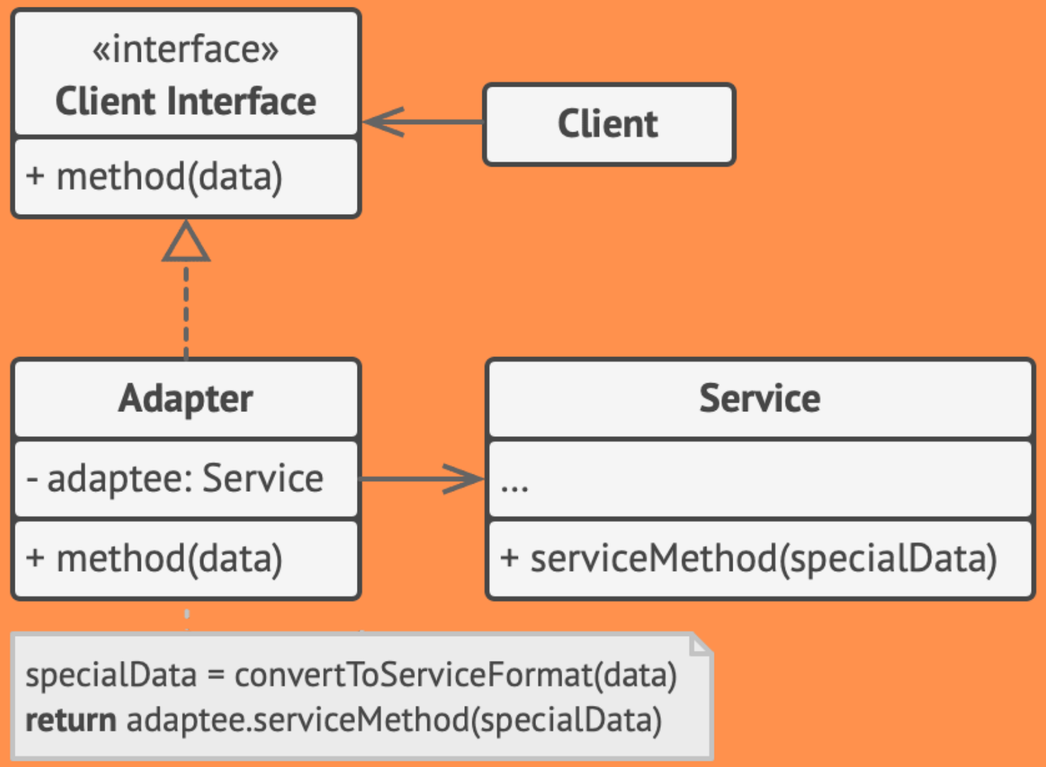
PROXY

- Es útil cuando se quiere controlar el acceso a un objeto o servicio.
- El proxy actúa como sustituto del servicio original y puede ejecutar acciones antes o después de una solicitud.
- El proxy y el servicio original comparten la misma interfaz, por eso se ven iguales ante quien los usa.
- El cliente debe usar esa misma interfaz, para que no note la diferencia entre el proxy y el objeto real.
- Así, el proxy puede intervenir sin que el cliente se dé cuenta.



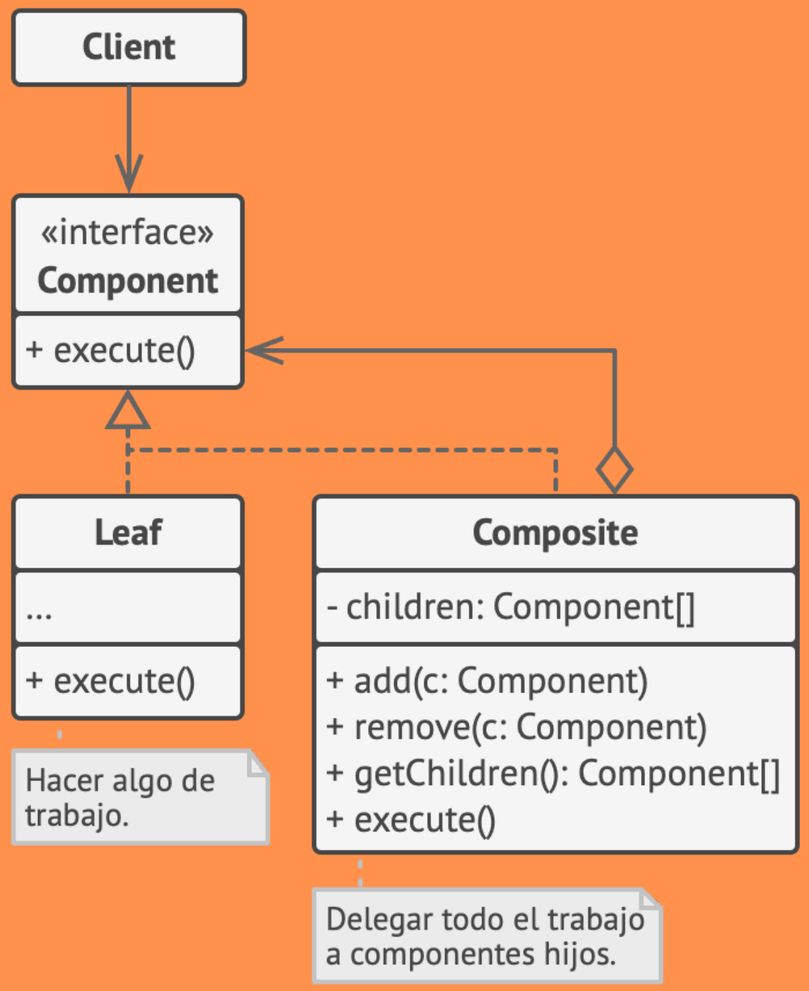
ADAPTER

- Sirve cuando un objeto o servicio tiene una interfaz incompatible con lo que necesita el cliente.
- El adapter envuelve al servicio original y adapta su interfaz para que sea compatible con la del cliente.
- Gracias al adapter, el cliente puede usar un servicio que antes no podía.
- La client interface define las reglas que deben seguir las clases para poder trabajar con el cliente.
- El adapter hereda de esta client interface para asegurarse de cumplir con lo que el cliente espera.



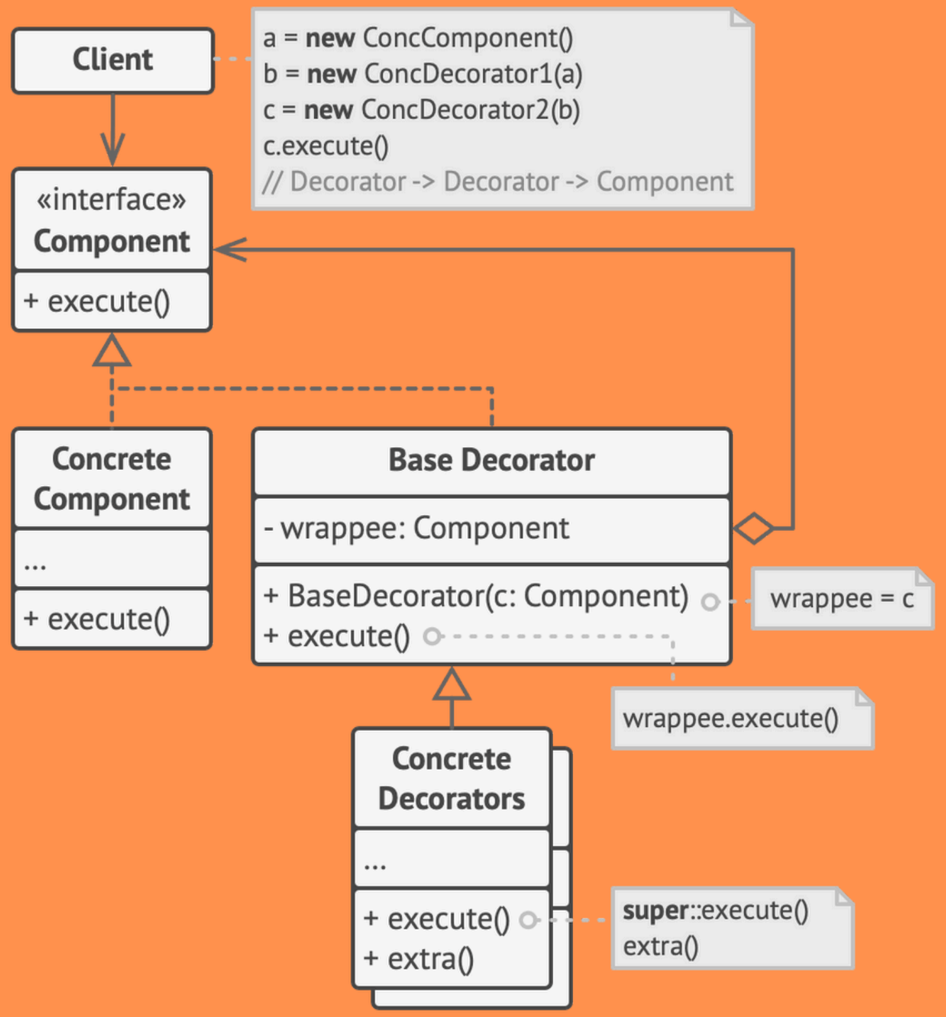
COMPOSITE

- Permite trabajar con objetos simples y compuestos usando una misma interfaz común.
- Los objetos leaf son unidades básicas con un comportamiento concreto.
- Los objetos composite están formados por otros objetos (leafs o composites) y les delegan tareas.
- Los composites pueden además procesar los resultados obtenidos de sus componentes.
- El cliente no necesita saber si está usando un leaf o un composite, ya que ambos siguen la misma interfaz.



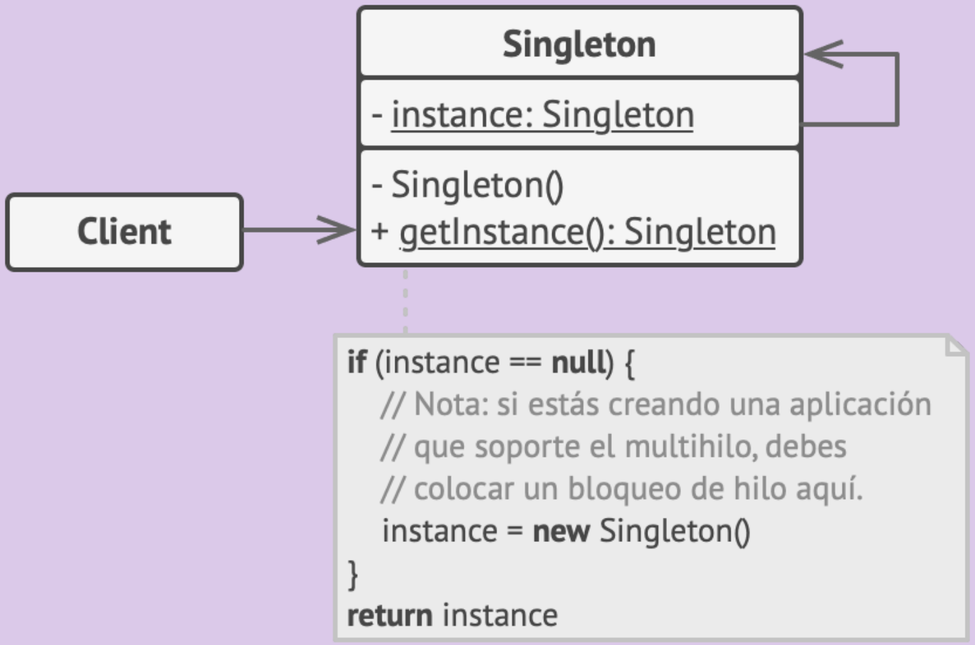
DECORATOR

- Es útil para agregar comportamiento adicional a un objeto sin modificar su estructura.
- Se usa una interfaz component que define cómo deben comportarse tanto los objetos base como los decoradores.
- El objeto base se llama concrete component.
- Los decoradores heredan de una clase base llamada base decorator, que también sigue la interfaz component.
- Los concrete decorators son clases específicas que implementan los cambios o mejoras deseadas.
- Para decorar un objeto, se crea un concrete decorator y se le pasa el concrete component como argumento.
- Cuando se llama un método del objeto decorado, el decorador primero llama al mismo método del objeto original y luego agrega su propio comportamiento.



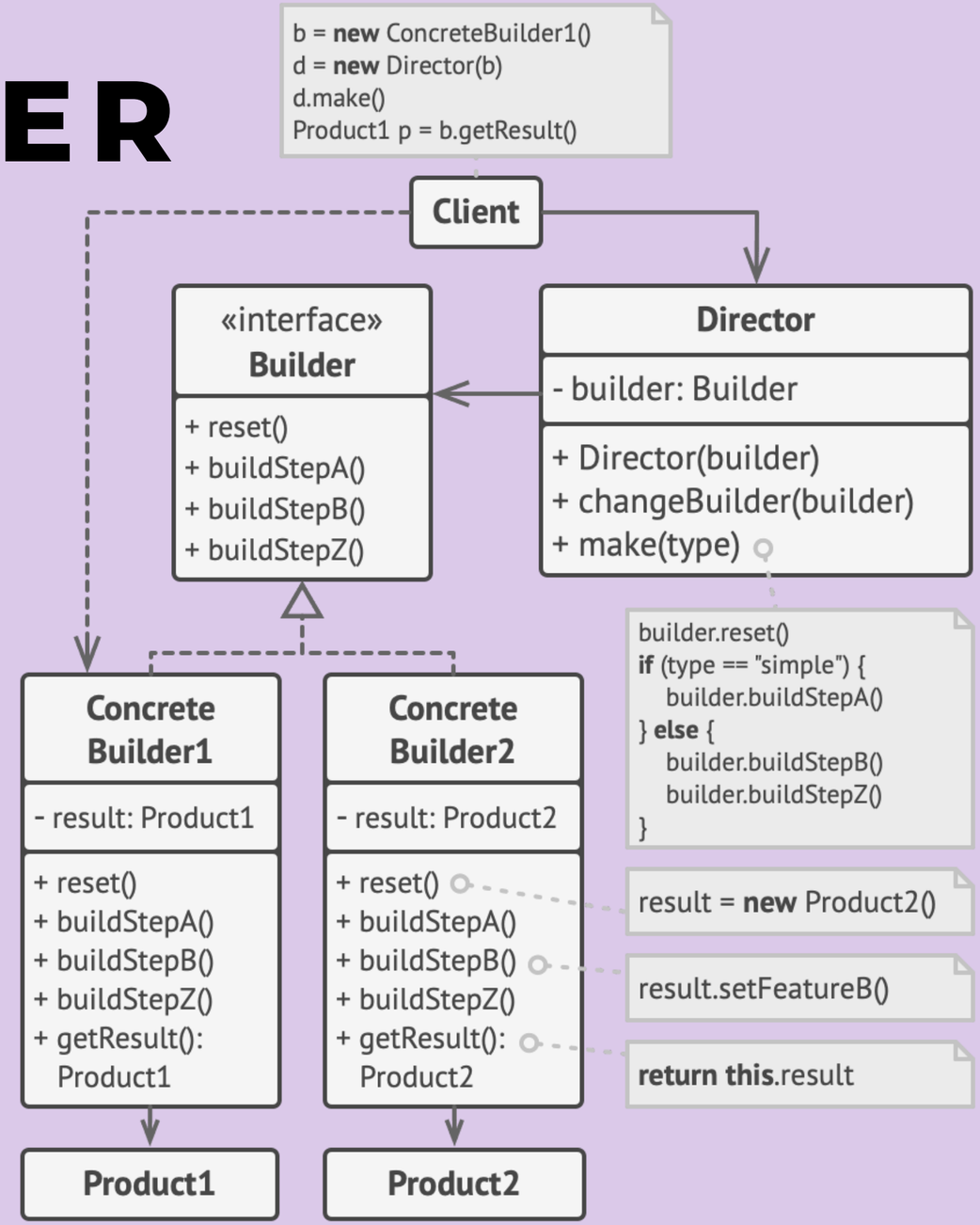
SINGLETON

- Se utiliza para asegurar que una clase tenga solo una única instancia.
- Esto convierte al objeto en una especie de variable global accesible desde cualquier parte del programa.
- El constructor de la clase se hace privado para evitar que se creen nuevas instancias directamente.
- En lugar de usar new, se accede al objeto a través de un método especial (como getInstance).
- Si ya existe una instancia, se retorna esa misma; si no, se crea una nueva.
- Así, todas las llamadas retornan siempre el mismo objeto compartido.



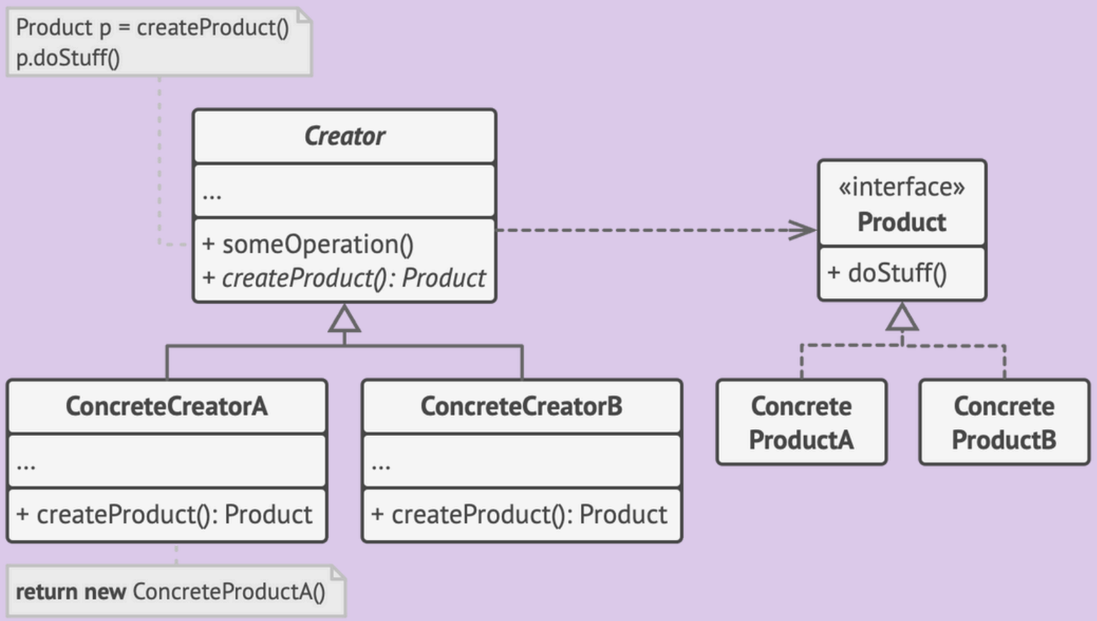
BUILDER

- Permite construir objetos paso a paso, dando más control sobre el proceso.
- Evita tener que crear una subclase distinta para cada combinación posible de características.
- También evita pasar muchos parámetros al inicializar el objeto.
- La interfaz constructora declara los métodos necesarios para construir las partes del objeto.
- Un concrete builder implementa estos métodos y se encarga de la construcción.
- El director no es obligatorio, pero ayuda a ocultar al cliente los detalles del proceso de construcción.
- El director define el orden en que deben ejecutarse los pasos, mientras que el concrete builder los lleva a cabo.



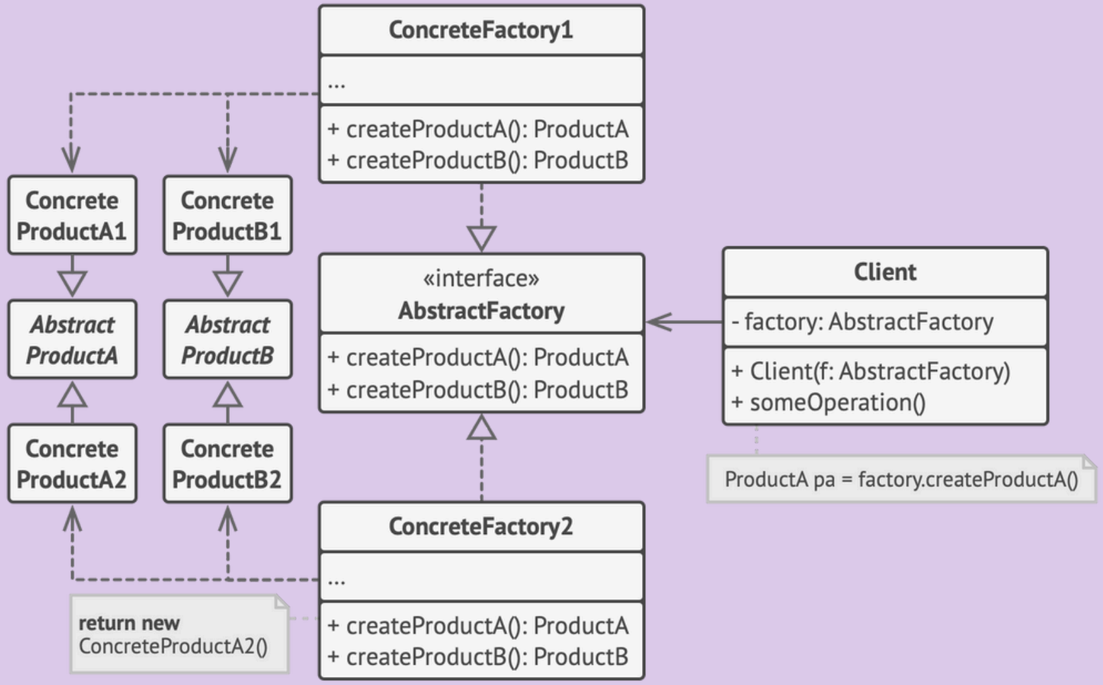
FACTORY METHOD

- Es útil para delegar la creación de objetos sin depender directamente de clases concretas.
- Existe una interfaz producto que define cómo deben comportarse los objetos a crear.
- Las subclases concretas implementan esta interfaz y definen el comportamiento real de los productos.
- Una clase fábrica (creator) declara un método que devuelve un objeto del tipo producto.
- Las fábricas concretas sobrescriben este método para decidir qué clase concreta crear.
- El cliente interactúa solo con la fábrica, sin saber qué clase específica está usando.
- Esto permite cambiar fácilmente la clase del producto sin modificar el código del cliente.



ABSTRACT FACTORY

- Es útil para crear familias de objetos relacionados sin depender de sus clases concretas.
- Se define una interfaz de fábrica abstracta que declara métodos para crear cada tipo de objeto de la familia (por ejemplo, botón, ventana, menú).
- Cada fábrica concreta implementa esta interfaz y crea objetos concretos que pertenecen a una misma familia (por ejemplo, todos con estilo Mac o Windows).
- Los objetos creados por una misma fábrica son compatibles entre sí y siguen un estilo coherente.
- El cliente solo crea una fábrica concreta y usa sus métodos para obtener los objetos necesarios.
- Esto garantiza consistencia entre los productos creados y permite cambiar de familia fácilmente sin modificar el código del cliente.



QUALITY ASSESMENT

¿QUE ES?

Asegurar que el producto cumpla con los requisitos (funcionales o no)

Verificaciones:
El producto hace lo que **se especificó**
Validaciones:
El producto hace lo que el **usuario necesita**

FORMAS DE LOGRAR LA CALIDAD

Evitar bugs al construir software (poco realista en productos grandes).
Análisis estático: revisión automática del código.
Inspección formal: revisión manual.
Testing: diseñar casos de prueba para encontrar errores.

NIVELES DE TESTING

- Unitario: prueba de componentes aislados (ej. clases, funciones).
- Integración: verificar cómo interactúan múltiples componentes.
- Sistema: prueba del sistema completo (end-to-end).
- Aceptación: validación por usuarios reales.

TESTING DE INTEGRACIÓN

Detecta errores que surgen al combinar módulos.

Estrategias:

- Big Bang: integrar todo al final.
- Bottom-Up: desde componentes pequeños a grandes.
- Top-Down: desde componentes grandes a pequeños (usando stubs).

INTEGRACIÓN CONTINUA

Testear constantemente al integrar nuevas funcionalidades.

Se realiza un build diario para mantener la calidad.

TESTING DE REGRESIÓN

Asegura que lo que ya funcionaba sigue funcionando después de un cambio.
Tipos:

- Tests de nuevas funcionalidades.
- Tests de funcionalidades modificadas.
- Casos representativos del sistema.

SMOKE & SANITY TESTS

Smoke Test: pruebas básicas tras una modificación para ver si el sistema puede correr.

Sanity Test: verificar que un error específico fue corregido.

AUTOMATIZACIÓN

Usar herramientas como Selenium para automatizar pruebas de regresión en aplicaciones web.

TESTS DE SISTEMA

Prueba del sistema completo.

Evalúa cumplimiento de requisitos funcionales y no funcionales.

LOAD & STRESS TESTS

Load Test: evaluar si el sistema resiste la carga esperada.

Stress Test: poner al sistema bajo carga extrema para ver su resistencia.

TESTS DE USABILIDAD

Observación de usuarios reales ejecutando tareas.

No mide estética ni preferencia, solo eficiencia de uso.

TESTS DE ACEPTACIÓN

Validados por usuarios reales.

Tipos:

- Alpha: en ambiente de desarrollo.
- Beta: en ambiente del cliente.
- Contract Acceptance: validación contractual.
- Regulation Acceptance: cumplimiento legal.
- Operational Acceptance: operación, mantenimiento, etc.

A/B TESTING

Comparar dos versiones distintas
del software (A y B).

Se mide cuál tiene mejor rendimiento
en una métrica definida.

ARQUITECTURA DE SOFTWARE

¿QUE ES?

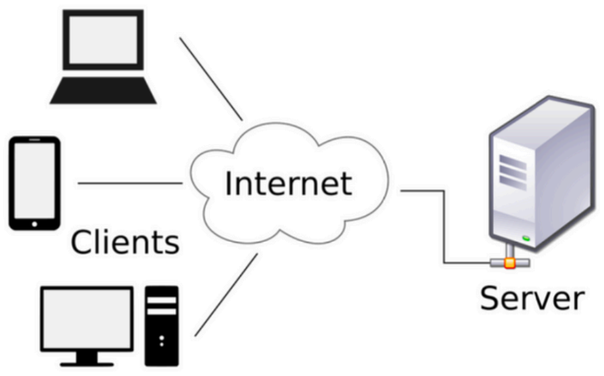
- Define la estructura del desarrollo de software.
- Permite anticipar decisiones costosas.
- Mejora la calidad haciendo explícitas las decisiones clave.
- Es fundamental para sistemas grandes, complejos y a largo plazo.

REQUISITOS DEL SOFTWARE MODERNO

- Mantenibilidad: debe mantenerse por años.
- Escalabilidad: soportar millones de usuarios.
- Confiabilidad: disponible 24/7.
- Eficiencia: baja latencia.

CLIENTE Y SERVIDOR

- Cliente: programa que envía solicitudes por internet.
- Servidor: programa que responde a esas solicitudes.
- Un servidor puede atender múltiples clientes al mismo tiempo.

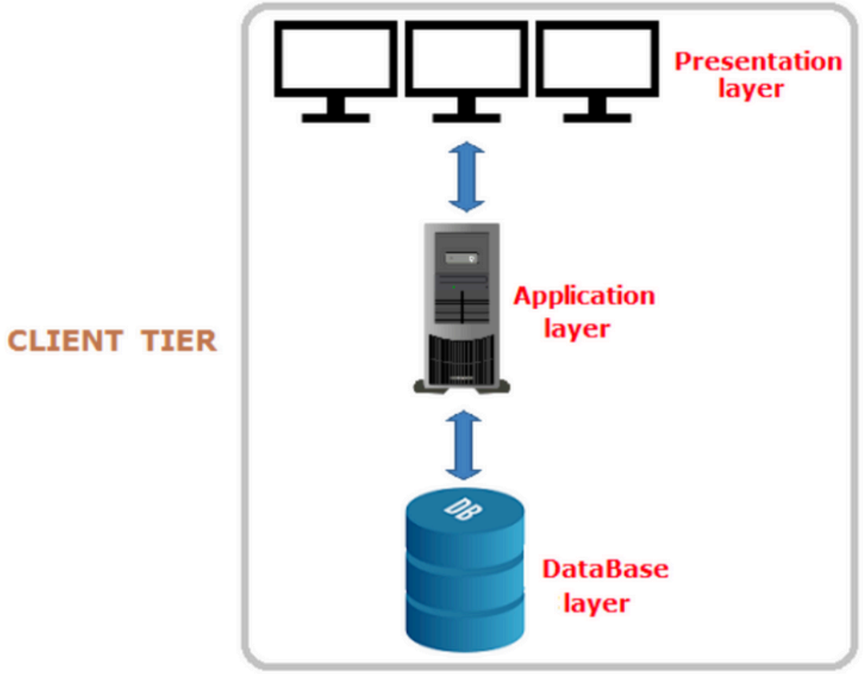


ARQUITECTURA POR CAPAS

- One-tier: todo en un solo sistema.
- Two-tier: cliente y base de datos separados.
- Three-tier: cliente, aplicación y base de datos distribuidos.
- Arquitectura moderna: incluye capas como interfaz, dominio, persistencia y datos.

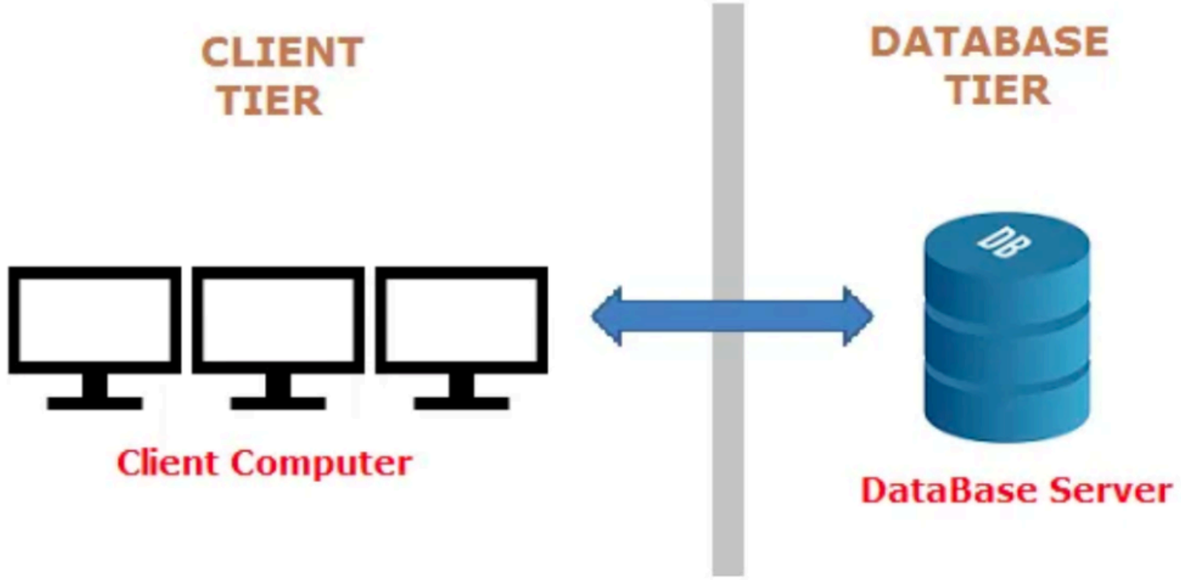
ARQUITECTURA POR CAPAS

ONE-TIER ARCHITECTURE

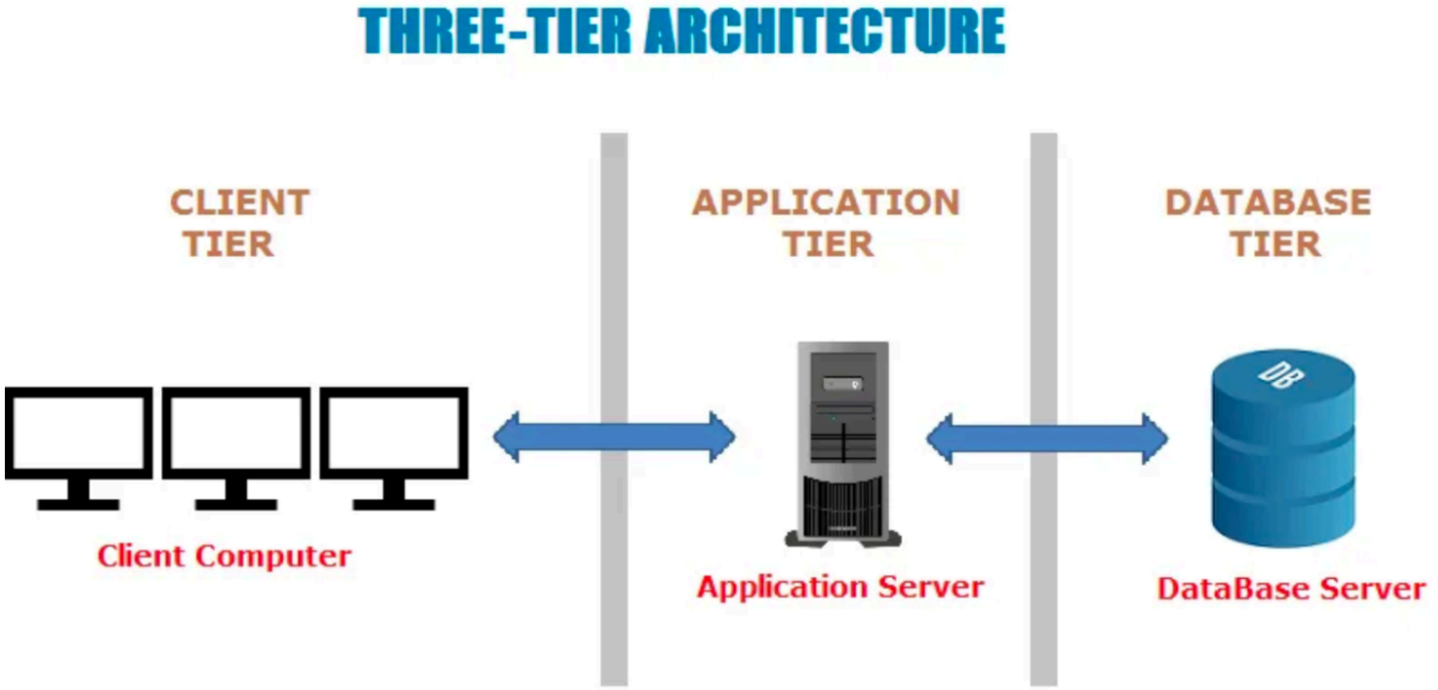


ARQUITECTURA POR CAPAS

TWO-TIER ARCHITECUTRE

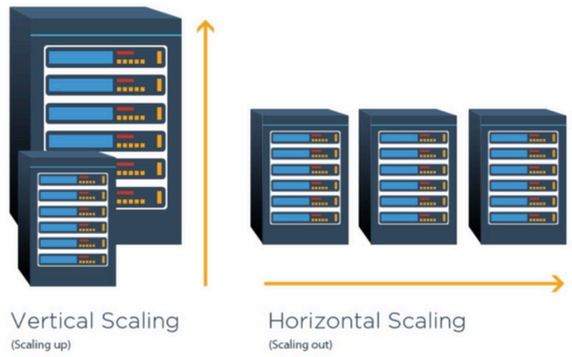


ARQUITECTURA POR CAPAS



ESCALABILIDAD

- Escalar verticalmente (más potente un solo servidor) o horizontalmente (más servidores).
- 100 o 1000 usuarios pueden funcionar bien, pero 1 millón requiere rediseño arquitectónico.



ARQUITECTURA ORIENTADA A SERVICIOS (SOA)

- Divide el sistema en servicios específicos (como cuenta, envío, reservas).
- Usa un bus de servicios para comunicar servicios con clientes.
- Cada servicio tiene su propia base de datos.

MICROSERVICIOS VS ENFOQUE TRADICIONAL

- Microservicios: servicios independientes, con su propia base de datos.
- Escalabilidad más sencilla al replicar servicios.
- Enfoque tradicional: todo junto, menos flexible para escalar.

CONTENEDORES Y DOCKERS

- Docker permite empaquetar aplicaciones con sus dependencias.
- Comparado con máquinas virtuales, los contenedores son más ligeros y portables.
- Dockerfile define cómo construir una imagen.

SCRUM

SCRUM

SCRUM es un marco de trabajo ágil para gestionar proyectos complejos y entregar productos de valor de manera incremental y colaborativa. Es ideal para ambientes caóticos o de alta incertidumbre.

ROLES EN SCRUM

Product Owner (PO):

- Define funcionalidades del producto.
- Establece el alcance, cronograma y prioridades del Product Backlog.
- Es responsable de alcanzar objetivos de negocio.
- Acepta o rechaza los resultados del Sprint.

ROLES EN SCRUM

Equipo de Desarrollo:

- Entre 4 a 8 personas (programadores, testers, diseñadores).
- Se autoorganiza (nadie asigna tareas).
- No hay jerarquías.

ROLES EN SCRUM

Scrum Master:

- Vela por los valores de SCRUM.
- Elimina impedimentos.
- Protege al equipo de interferencias externas.
- Ayuda a mejorar la productividad.

ARTEFACTOS Y EVENTOS SCRUM

1. Product Backlog
   - Lista priorizada de todo lo que se desea en el producto.
   - Evoluciona constantemente.

2. Sprint Planning
   - Reunión al comienzo del Sprint.
   - Se seleccionan ítems del Product Backlog para formar el Sprint Backlog.

ARTEFACTOS Y EVENTOS SCRUM

3. Sprint Backlog
   - Conjunto de tareas para ese sprint.
   - Cada miembro elige la tarea que hará.
   - Se actualiza diariamente.
   - Cualquiera puede modificarlo.

4. Sprint
   - Ciclo fijo de desarrollo (2 a 4 semanas).
   - Al final se entrega un incremento funcional del producto.

ARTEFACTOS Y EVENTOS SCRUM

5. Daily Scrum
   - Reunión diaria breve.
   - Cada miembro responde:
     a. ¿Qué hice ayer?
     b. ¿Qué haré hoy?
     c. ¿Tuve algún impedimento?

6. Sprint Review
   - Se presenta el incremento al PO y stakeholders.
   - Se revisan logros y posibles ajustes.

ARTEFACTOS Y EVENTOS SCRUM

7. Sprint Retrospective
   - Reunión interna para mejorar el proceso.
   - ¿Qué funcionó bien?, ¿qué se puede mejorar?

DISEÑO

DIAGRAMAS EXISTENTES

Diagrama de Clases

Diagrama de Secuencia

DIAGRAMA DE CLASES

DIAGRAMA DE CLASES

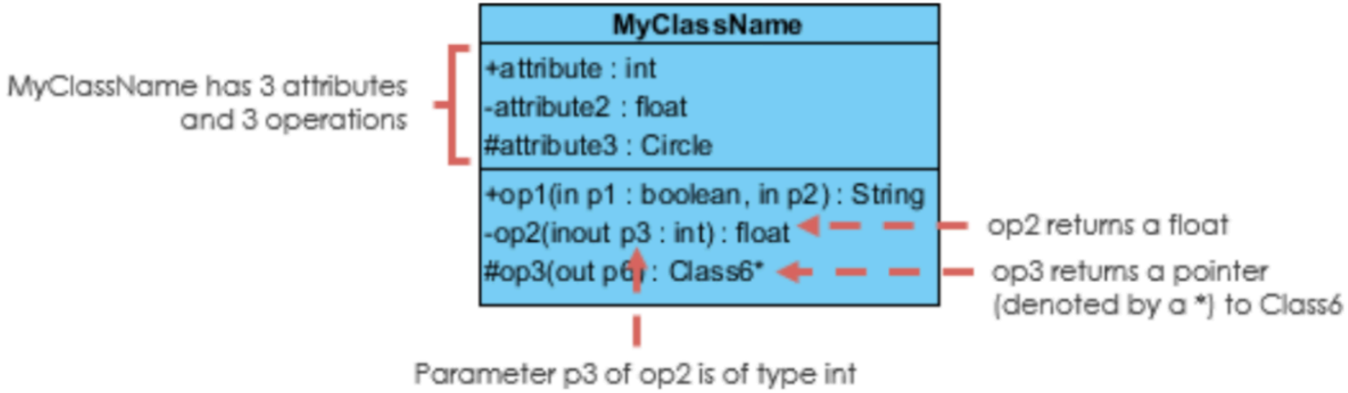
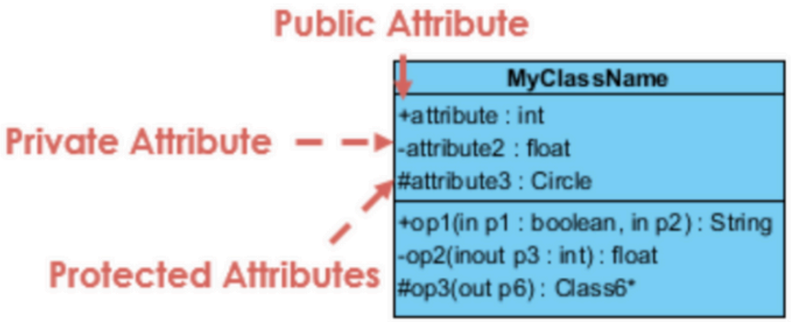
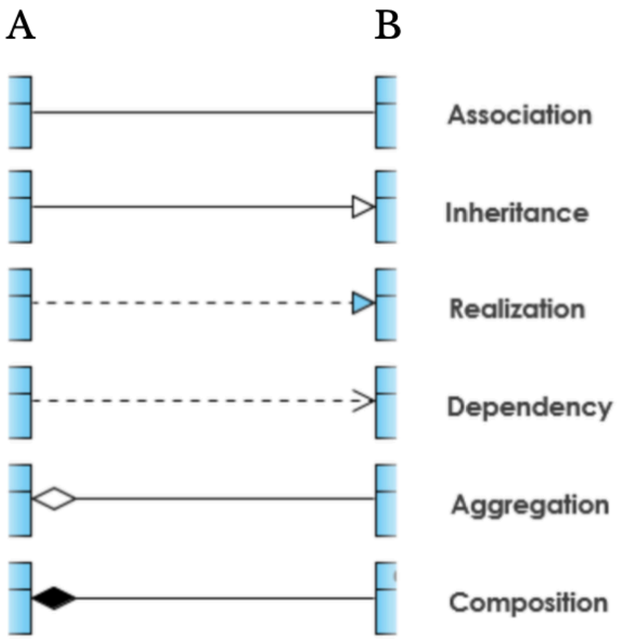


DIAGRAMA DE CLASES



- **Público (+)**: puede ser accedido desde cualquier clase del sistema.
- **Protegido (#)**: solo puede ser accedido desde la misma clase y clases hijas.
- **Privado (-)**: solo puede ser accedido desde la misma clase.

DIAGRAMA DE CLASES (RELACIONES)



- **Asociación:** Un objeto de de la clase referencia otro por mucho tiempo (atributo)
- **Herencia:** Una clase hereda de otra
- **Realización:** Una clase implementa una interfaz o módulo
- **Dependencia:** Un objeto de la clase referencia a otro de forma **TEMPORAL**
- **Agregación:** Un objeto de la clase A usa objetos de la clase B
- **Composición**: Un objeto de la clase A **CREA** y usa objetos de la clase B

DIAGRAMA DE CLASES (CARDINALIDADES)

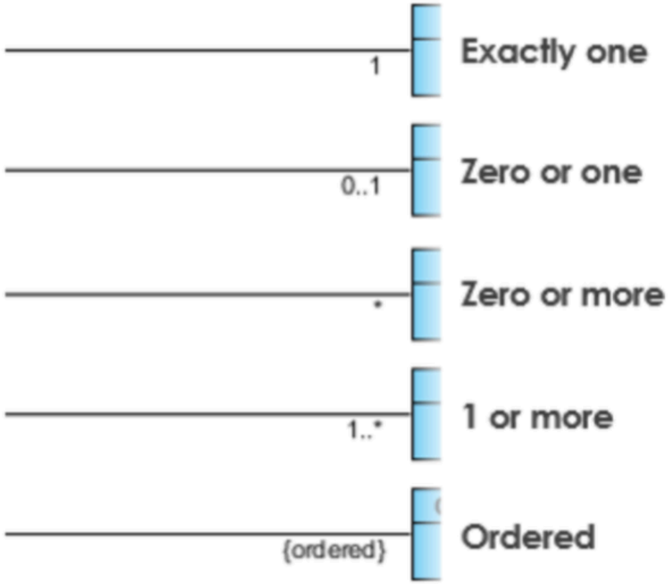
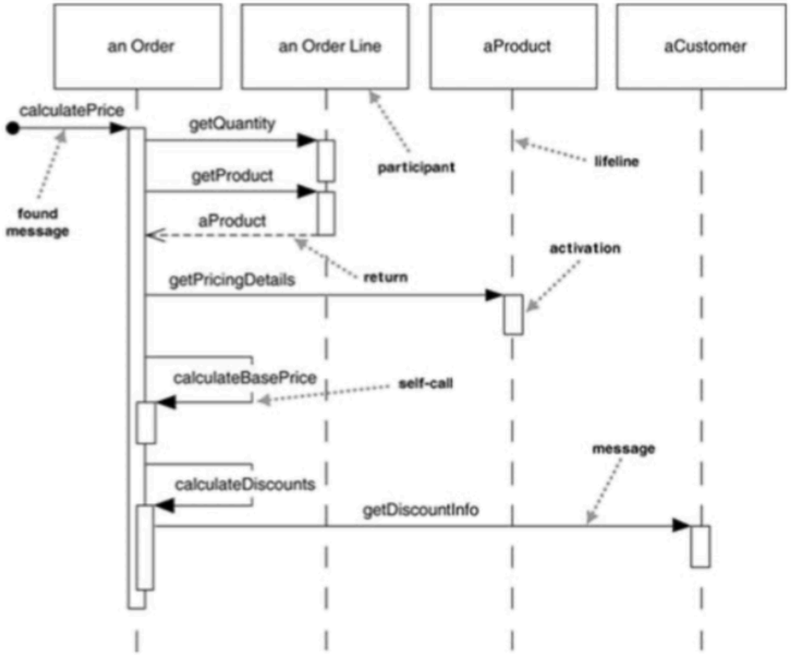
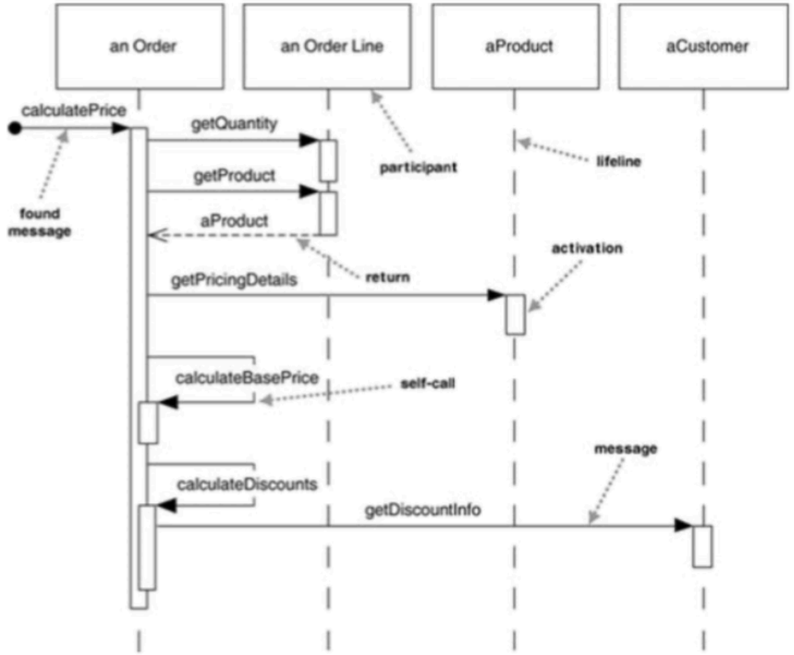


DIAGRAMA DE SECUENCIA



- **Cada Caja** es la instancia de la clase
- **La linea Punteada (no olvidar)** es la **línea de vida** de un objeto
- Se lee de **arriba hacia abajo**
- Mientras el objeto llame sub-funciones de otros objetos, el primero **permanece vivo**

DIAGRAMA DE SECUENCIA

```
class Order
     def calculatePrice()

        a = orderLine.getQuantity()
        b = orderLine.getProduct()
        product.getPricingDetails()
        self calculateBasePrice()
        self calculateDiscount()
        customer.getDiscountInfo()

    end
end
```